Property-based testing

with ScalaCheck

ThoughtWorks®

The most under appreciated practice from PF

Too much talks about property-based testing is barely enough

What is it?

```
def Sum(x: Int, y: Int) = ???
```

```
def sumOfZeros = Sum(0, 0) must_== 0
def sumZeroAndANumber = Sum(0, 1) must_== 1
def sum = Sum(2, 2) must_== 4
```

```
property("sum") = forAll{ (x: Int, y: Int)
    Sum(x, y) == x + y
}
```

```
property("square root") =
        forAll{(x: Int) => Math.sqrt(x * x) == x }
```

```
> test
[info] ! Math.square root: Falsified after 0 passed tests.
[info] > ARG_0: -1
[info] > ARG_0_ORIGINAL: -1442281511
[info] Failed: Total 1, Failed 1, Errors 0, Passed 0
[error] Failed tests:
[error]                 SquareRootSpec
```

```
property("square root") =
  forAll{(x: Int) => Math.sqrt(x * x) == Math.abs(x) }
```

```
> test
[info] ! Math.square root: Falsified after 2 passed tests.
[info] > ARG_0: 46341
[info] > ARG_0_ORIGINAL: 633604525
[info] Failed: Total 1, Failed 1, Errors 0, Passed 0
[error] Failed tests:
[error]         SquareRootSpec
```

Shrinking

```
scala> implicitly[Shrink[Int]].shrink(2095894375).iterator
res0: Iterator[Int] = non-empty iterator


scala> res0.next()
res1: Int = 0

scala> res0.next()
res2: Int = 1047947188

scala> res0.next()
res3: Int = -1047947188

scala> res0.next()
res4: Int = 1571920782
```

```
property("square root") =
  forAll("X" |: arbitrary[Int]) { x => Math.sqrt(x * x) == Math.abs(x)}
```

```
> test
[info] Compiling 1 Scala source to /Users/sfilimon/codemesh/
check-all/target/scala-2.11/test-classes...
[info] ! Math.square root: Falsified after 0 passed tests.
[info] > X: 46341
[info] > X_ORIGINAL: 2095894375
[info] Failed: Total 1, Failed 1, Errors 0, Passed 0
[error] Failed tests:
[error]             SquareRootSpec
```

```
scala> Int.MaxValue
res0: Int = 2147483647

scala> 46341*46341
res1: Int = -2147479015
```

```
`*`::Int => Int => Int
```

```
property("square root") = forAll{ (x: Int) =>
  Checked.option(x * x).isDefined ==> (
    Math.sqrt(x * x) == Math.abs(x))
```

```
> test
[info] + Math.square root: OK, passed 100 tests.
[info] Passed: Total 1, Failed 0, Errors 0, Passed 1
[success] Total time: 0 s, completed 31/10/2016 11:16:01 PM
```

```
property("square root") = forAll{ (x: Int) =>
  Checked.option(x * x).isDefined ==>
   collect(x) {
     Math.sqrt(x * x) == Math.abs(x)
    }
}
```

```
> test
[info] + Math.square root: OK, passed 100 tests.
[info] > Collected test data:
[info] 35% -1
[info] 34% 1
[info] 31% 0
[info] Passed: Total 1, Failed 0, Errors 0, Passed 1
```

The condition for the property is too strict

```
property("square root") = forAll{ (x: Int) =>
  (Checked.option(x * x).isDefined && x > 0) ==>
   collect(x) {
     Math.sqrt(x * x) == Math.abs(x)
   }
}
```

```
> test
[info] ! Math.square root: Gave up after only 67 passed tests. 501 tests were discarded.
[info] > Collected test data:
[info] 100% 1
[info] Failed: Total 1, Failed 1, Errors 0, Passed 0
[error] Failed tests:
[error]         SquareRootSpec
```

```
val smallValues = Gen.choose(-46340, 46340)
property("square root") = forAll(smallValues){ x =>
    collect(x) {
      Math.sqrt(x * x) == Math.abs(x)
    }
}
```

```
> test
[info] + Math.square root: OK, passed 100 tests.
[info] > Collected test data:
[info] 1% -16826
[info] 1% 4323
[info] 1% 42927
[info] 1% 16213
[info] 1% -10636
…
```

ScalaCheck property for square root

Shrinking

With restricted/conditioned generator

With customised generator

Validated generated input

Discover Edge cases over inventing them

Gen[T]

```
scala> val intGen = arbitrary[Int]

scala> intGen.sample
res10: Option[Int] = Some(0)

scala> intGen.sample
res11: Option[Int] = Some(-391553415)

scala> intGen.sample
res12: Option[Int] = Some(-871817096)


scala> intGen.sample
res13: Option[Int] = Some(-9882328)
```

Creating generators

```
case class Customer(name: String, age: Int)
```

Monadic composition

```
val customerGen: Gen[Customer] =
  arbitrary[String].flatMap(name =>
  arbitrary[Int].map(age => Customer(name, age)))
```

Creating generators

```
case class Customer(name: String, age: Int)
```

Monadic composition

```
val customerGen: Gen[Customer] = for {
  name             ← arbitrary[String]
  age              ← arbitrary[Int]
} yield Customer(name, age)
```

```scala
case class Insurance(provider: String)

case class Customer(name: String,
                    age: Int,
                    insurance: Option[Insurance])
```

```scala
implicit val insuranceGen =
   Arbitrary(arbitrary[String].map(Insurance))
```

```scala
val customerGen: Gen[Customer] = for {
  name      ← arbitrary[String]
  age       ← arbitrary[Int]
  insurance ← arbitrary[Option[Insurance]]
} yield Customer(name, age, insurance)
```

Creating generators

```
case class Customer(name: String,
                    age: Int,
                    insurance: Option[Insurance])
```

Applicative composition:

```
val customerGen: Gen[Customer] =
   (arbitrary[String]
     |@| arbitrary[Int]
       |@| arbitrary[Option[Insurance]])
     .map { (name, age, insurance) =>
         Customer(name, age, insurance)}
```

Creating generators

```
val customerGen: Gen[Customer] = for {
  name           ← arbitrary[String]
  age            ← arbitrary[Int]
  insurance      ← arbitrary[Option[Insurance]]
} yield Customer(name, age, insurance)
```

```
val customerGen: Gen[Customer] = for {
  name           ← arbitrary[String]
  age            ← arbitrary[Int]
  insurance      ← arbitrary[Insurance]
  maybeInsurance ← Gen.oneOf(None, Some(insurance))
} yield Customer(name, age, maybeInsurance)
```

scalacheck-shapeless

```
case class Customer(name: String, age: Int)
```

```
val customerGen: Gen[Customer] = for {
  name             ← arbitrary[String]
  age              ← arbitrary[Int]
} yield Customer(name, age)
```

```
implicitly[Arbitrary[Customer]]
```

Restricting and reshaping generators

```
property("square root") = forAll{ (x: Int) =>
  Checked.option(x * x).isDefined ==> { … }
```

```
val smallValues = org.scalacheck.Gen.choose(-46340, 46340)
```

Restricting and reshaping generators

```
case class Customer(name: String,
                    age: Int,
                    insurance: Option[Insurance])
```

```
val customerGen: Gen[Customer] = for {
  name           ← arbitrary[String]
  age            ← arbitrary[Int]
  insurance      ← arbitrary[Option[Insurance]]
} yield Customer(name, age, insurance)
```

```
val customersWithoutInsurance =
  customerGen.map(_.copy(insurance = None))
```

Restricting and reshaping generators

```
val customerWithInsurance = arbitrary[Customer]
        .suchThat(_.insurance.isDefined) // ==>
```

```
val customerWithInsurance = arbitrary[Customer]
        .retryUntil(_.insurance.isDefined)
```

Utilities of org.scalacheck.Gen

```
age     ← choose(0, Int.MaxValue)
```

```
insurance       ← arbitrary[Insurance]
maybeInsurance  ← oneOf (None, Some(insurance))
```

Utilities of org.scalacheck.Gen

name ← **alphaStr**

insuranceNumber ← **numStr**

numForSquareRoot ← **posNum**

Shrinking

```
scala> val stringShrink =
implicitly[Shrink[String]].shrink("hello world").iterator
stringShrink: Iterator[String] = non-empty iterator

scala> stringShrink.next
res6: String = hello

scala> stringShrink.next
res7: String = " world"

scala> stringShrink.next
res8: String = he world

scala> stringShrink.next
res9: String = hello wo
```

```
scala> implicitly[Shrink[(String, Int)]].shrink("fo",
-10).force

res19: scala.collection.immutable.Stream[(String, Int)] =
Stream((f,-10), (o,-10), (?o,-10), (3o,-10),
(Ã”Ã¸Ã§o,-10), (Mo,-10), (Ã”Ã¦óo,-10), (Zo,-10),
(Ã”Ã¦Â¶o,-10), (`o,-10), (Ã”Ã¦ào,-10), (co,-10),
(Ã”Ã¦Ã¹o,-10), (eo,-10), (Ã”Ã¦Ãµo,-10), (f?,-10),
(f8,-10), (fÃ”Ã¸È,-10), (fT,-10), (fÃ”Ã¦Â¨,-10),
(fb,-10), (fÃ”Ã¦Ã»,-10), (fi,-10), (fÃ”Ã¦Ã³,-10),
(fl,-10), (fÃ”Ã¦Ã®,-10), (fn,-10), (fÃ”Ã¦Ò,-10), (fo,0),
(fo,-5), (fo,5), (fo,-8), (fo,8), (fo,-9), (fo,9))
```

```scala
case class OptimalProposition(name: String, score: Int)
```

```scala
implicit val arbOptimalProposition = Arbitrary(for {
  name <- arbitrary[String]
  score <- arbitrary[Int]
} yield OptimalProposition(name, score))
```

```scala
property("optimal proposition") =
  forAll {(proposition: OptimalProposition) =>
     proposition.score > 0}
```

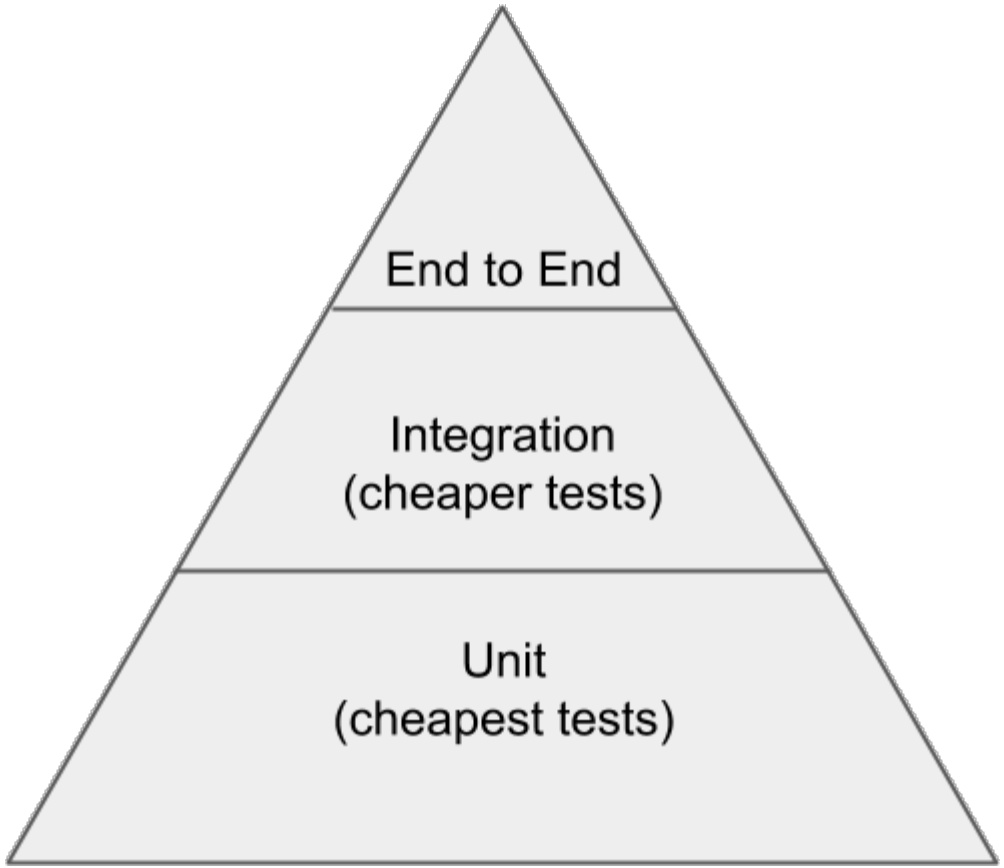
```
> test
[info] ! String.optimal proposition: Falsified after 5 passed
tests.
[info] > ARG_0: OptimalProposition(,-2090196921)
[info] Failed: Total 1, Failed 1, Errors 0, Passed 0
```

```
implicit val shrinkOptimalPropositions:
Shrink[OptimalProposition] =
  Shrink { proposition =>
    implicitly[Shrink[(String, Int)]].
      shrink((proposition.name, proposition.score))
      .map((OptimalProposition.apply _).tupled)
  }
```

```
> test
[info] Compiling 1 Scala source to /Users/sfilimon/codemesh/
gen-magic/target/scala-2.11/test-classes...
[info] ! String.optimal proposition: Falsified after 0 passed
tests.
[info] > ARG_0: OptimalProposition(,0)
[info] > ARG_0_ORIGINAL: OptimalProposition(,-1435018676)


> test
[info] ! String.optimal proposition: Falsified after 5 passed
tests.
[info] > ARG_0: OptimalProposition(,-2090196921)
[info] Failed: Total 1, Failed 1, Errors 0, Passed 0
```

Shrinking with shapeless scalacheck

```
import org.scalacheck.Shapeless._

case class OptimalProposition(name: String, score: Int)

property("optimal proposition") = forAll {(proposition:
OptimalProposition) =>
    proposition.score > 0}
```



Example based tests

Property-based tests

Property-based tests

Useful patterns: roundtrip

```
property("reverse") = forAll { (v: String) => v.reverse.reverse == v }
```

Useful patterns: roundtrip

```
prop { decisionId: DecisionId =>
  DecisionId.fromCanonical(
    decisionId.toCanonical) must_== decisionId}
```

Useful patterns: roundtrip

```
val activityClassJson: Gen[JsObject] = ???

val jsonRoundTrip = forAll(activityClassJson) { input ⇒
  val activity: ActivityClass = input.as[ActivityClass]
  Json.toJson(activity).as[ActivityClass] must_== activity
}
```

Useful patterns: roundtrip

```
val roundtrip = prop {event: Event =>
  withDb { database =>
    EventsSink(database).insert(event)
    EventsSink.selectRecent must contain(event)
  }
}
```

Useful patterns: invalid input

```
var invalidCase = forAll(arbInvalidCsvFile) { input =>
  CSVParser(input) must be_-\/
}
```

Generate functions

```
scala> val fa = arbitrary[Int => String].sample.get
fa: Int => String = <function1>

scala> fa(3)
res22: String = 擦鱉靹咤鮞鶬㊣醼黿岬ル떬噂聶豹?ঌ惆诶

scala> fa(3)
res23: String = 擦鱉靹咤鮞鶬㊣醼黿岬ル떬噂聶豹?ঌ惆诶

scala> fa(4)
res24: String = 鼇?瞋?流酮糊骄叟貓鏺孍?囤兟
```

Spec2 + ScalaCheck = <3

```
class IntSumSpec extends Specification with ScalaCheck { def is = s2"""

  The IntSum Monoid must respect Monoid laws
    left identity                                          $leftid
    right identity                                         $rightid
    associativity                                          $associativity
                                                           """

  def leftid  = prop { i: Int => 0 |+| i === i }
  def rightid = prop { i: Int => i |+| 0 === i }

  def associativity = prop { (a: Int, b: Int, c: Int) =>
    ((a |+| b) |+| c) === (a |+| (b |+| c))
  }

}
```

Other languages

```
describe Fizzbuzz do
    it 'do fizzbuzz very hard' do

    end
end
```

```
describe Fizzbuzz do
    it 'do fizzbuzz very hard' do
        property_of { integer }.
        check { |i|

        }
    end
end
```

```
describe Fizzbuzz do
    it 'do fizzbuzz very hard' do
        property_of { integer }.
        check { |i|
            if(i % 3 == 0 && i % 5 == 0)
                expect(Fizzbuzz.do(i)).to eq('fizzbuzz')
            end
        }
    end
end
```

```
describe Fizzbuzz do
    it 'do fizzbuzz very hard' do
        property_of { integer }.
        check { |i|
            if(i % 3 == 0 && i % 5 == 0)
                expect(Fizzbuzz.do(i)).to eq('fizzbuzz')
            else(i % 3 == 0)
                expect(Fizzbuzz.do(i)).to eq('fizz')
            end
        }
    end
end
```

```
describe Fizzbuzz do
    it 'do fizzbuzz very hard' do
        property_of { integer }.
        check { |i|
            if(i % 3 == 0 && i % 5 == 0)
                expect(Fizzbuzz.do(i)).to eq('fizzbuzz')
            elsif(i % 3 == 0)
                expect(Fizzbuzz.do(i)).to eq('fizz')
            else (i % 5 == 0)
                expect(Fizzbuzz.do(i)).to eq('buzz')
            end
        }
    end
end
```

```
describe Fizzbuzz do
    it 'do fizzbuzz very hard' do
        property_of { integer }.
        check { |i|
            if(i % 3 == 0 && i % 5 == 0)
                expect(Fizzbuzz.do(i)).to eq('fizzbuzz')
            elsif(i % 3 == 0)
                expect(Fizzbuzz.do(i)).to eq('fizz')
            elsif (i % 5 == 0)
                expect(Fizzbuzz.do(i)).to eq('buzz')
            else
                expect(Fizzbuzz.do(i)).to eq(i)
            end
        }
    end
end
```

```
class Fizzbuzz
    def self.do number
        if(number % 3 == 0 && number % 5 == 0)
            'fizzbuzz'
        elsif(number % 3 == 0)
            'fizz'
        elsif (number % 5 == 0)
            'buzz'
        else
            number
        end
    end
end
```

```
pub mod peasant {
  pub fn peasant(a: usize, b: usize) -> Option<usize> {
    …
  }
```

```rust
#[test]
fn test_peasant() {
}
```

```
#[test]
fn test_peasant() {
 quickcheck(peasant_is_as_multiplication as
     fn(usize, usize) -> bool)
}
```

```
#[test]
fn test_peasant() {
  fn peasant_is_as_multiplication(a: usize, b:usize) -> bool
   {
    }

 quickcheck(peasant_is_as_multiplication as
    fn(usize, usize) -> bool)
}
```

```rust
#[test]
fn test_peasant() {
  fn peasant_is_as_multiplication(a: usize, b:usize) -> bool
   {
    peasant::peasant(a, b) == a.checked_mul(b)
   }

 quickcheck(peasant_is_as_multiplication as
    fn(usize, usize) -> bool)
}
```

Thank you!